

Catching what the gateway misses

Options after the anomaly model, with what each would cost — Intelligent API Security Gateway, 9 September 2026

The short answer

Yes, there is a better option than more machine learning, and it is simpler. The attacks the gateway misses have two clear fingerprints: scanners hit pages that do not exist, and slow password-guessers keep failing for many minutes in a row. Neither needs a model — each is a short rule. On our data those rules catch **65 of the 76** attack-minutes the gateway currently misses, and flag **0 of 13,572** normal minutes.

The anomaly model is still worth keeping, but for a narrower job: it catches the **10 remaining minutes** — the messy first minutes of a slow attack, before any rule has enough history. Rules plus model: **75 of 76**.

Recommended: build the rules first (about three days), keep the existing model as a second layer, and stop spending time on feature engineering for the model. Details and the reasoning below.

1. Where we are

What a "minute" means in this document. The dataset slices traffic into one-minute chunks per user. One row = one user, one minute. An attacker active for ten minutes is ten rows. Every count below is a count of those rows.

The target. In the test portion of dataset v4 there are **76 attack-minutes where the gateway did nothing** — no detector fired, nothing was blocked. These are the gaps the anomaly layer exists to fill. Everything else the gateway already handles, so a model that re-detects those is not adding anything.

76

attack-minutes the gateway missed (test split)

64

of them are one attack: slow path scanning
(`low_and_slow_enumeration`)

10

are the opening minutes of a slow password-guesser
(`slow_brute_force`)

What has been tried. Three models on the fresh v3/v4 data, all Isolation Forest, all evaluated the same way (1% false-positive budget per user type, threshold picked on validation, test looked at once):

Model	Overall attack recall	False positives	Missed-minutes caught	Note
Baseline, 13 features	5.6%	0.12%	10 / 76	All 10 are slow-brute-force opening minutes
+ login-regularity feature (×10)	27.4%	0.16%	0 / 76	Only re-catches attacks the gateway already stopped
+ unmatched-route feature (v4)	6.4%	0.16%	10 / 76	Feature is perfect in the data; model cannot use it

The second row is the trap worth remembering: a model can look five times better on the headline number and be worse at the only job that matters. *Overall recall* counts every attack-minute, most of which the gateway already catches. *Missed-minutes caught* is the honest number.

2. The finding that changes the plan

While adding the third model we found a clean signal for the scanning attacks: the share of a user's requests that go to **pages that do not exist in the app at all**. Across all 23 collection runs:

Traffic	Requests to non-existent pages
Path traversal attack	99.9%
Fast scanning attack	54.9%
Slow scanning attack	53.6%
Every normal user type, including the one that deliberately hits dead links	0.0% (0 of 121,480 requests)

The dead-link user stays at zero because they open a *real* page with a bad ID. A scanner opens pages the app has never served. That is the difference between a lost person and a probe, and it is exact.

Why the model still scored 0% on scanning after we added it. The model only ever trains on normal traffic — that is what "anomaly detection" means. In normal traffic this number is always exactly zero: 8,883 training rows, every one of them 0.000. A model cannot learn a dividing line from a column that never moves. So when a scanner arrives at 0.54, nothing in the model says that is unusual.

This is not a bug and it is not fixable by tuning. It is a property of the approach: **the better a feature separates attackers from normal users, the flatter it is in normal traffic, and the more invisible it is to a model trained only on normal traffic.** The features that matter most are precisely the ones this kind of model cannot see. That is the reason to look beyond it.

3. The options

Complexity scale used below. **LOW** one to three days, one or two files, follows an existing pattern. **MEDIUM** about a week; needs new state or a new module. **HIGH** weeks; new architecture, unclear payoff. Every "catches" number is measured on dataset v4 unless it says otherwise.

Option 1 — Behavioural rules in the Go detectors **LOW**

Three rules, each one sentence. Simulated on v4 tonight; not yet written in Go.

Rule	What it catches	Missed minutes	Normal minutes flagged
A. More than 30% of this minute's requests hit non-existent pages	Scanners, traversal	54 / 76	0 / 13,572
B. Any request to a non-existent page in the last 5 minutes	The slowest scanners (5 requests a minute, some minutes all "real" pages)	+10	0 / 13,572
C. Three consecutive minutes where every login failed	Slow password-guessing, from its third minute on	+1	0 / 13,572
All three together		65 / 76	0

Why rule C is three minutes and not one. A one-minute version ("every login this minute failed") flags the forgetful user 137 times — that persona exists in the traffic mix precisely to expose this mistake. Measured across 184 forgetful-user sessions, the longest run of all-failed minutes is **2**. Every slow-brute-force session runs for **10 or more**. A threshold of three separates them with room to spare. The cost is that the first two minutes of a real attack are not flagged by the rule — which is exactly the gap the model fills (Option 2).

Why these are hard to evade. Rules are visible to an attacker, so it is fair to ask whether they can be tuned around. To dodge rule A a scanner must only request pages that exist — which is no longer scanning. To dodge rule C an attacker must stop failing after two minutes and wait — which slows the attack down, and slowing attackers down is the point. Each rule encodes something the attack *must* do, not a detail of how our test attacks happen to be written. That is the argument that they will hold on attacks we did not write.

Fit with the architecture. Exact. The team rule is "detectors gather evidence, the decision engine decides"; these are detectors. They are deterministic, so they can be unit-tested and their decisions can be explained line by line, which the design document requires. Rule A is a behavioural version of the existing scanner detector, which today only knows a fixed list of bad paths like `/.env` and `/wp-admin` — an attacker just uses different paths.

Work. Extend `enumeration_path_traversal.go` with the ratio and a short per-address rolling window; extend `brute_force.go` with a per-address consecutive-minute counter. Both need a little state per address with expiry, in memory or Redis, in the same style as the existing detectors. Thresholds go in `config.yaml`. Tests for each rule and for the forgetful-user case specifically. Roughly three days.

Risk. Low. The main one is thin data — 76 minutes is not many. See section 5.

Verdict: do this first.

Option 2 — Keep the existing Isolation Forest as a second layer LOW

Already built, already loadable by the runtime. No new work beyond deploying it.

The baseline model caught **10 of 10** slow-brute-force opening minutes that the gateway missed — the ragged first minutes with four requests and irregular timing, which no rule can see yet because there is no history. It did this at 0.12% false positives. That is what a good anomaly model is for: shapes nobody wrote a rule for.

What not to do. Stop engineering features for it. Tonight showed that both engineered features — login regularity and unmatched routes — made the model worse or no better on the minutes that matter, for two different structural reasons (one rewards regular timing and so misses ragged minutes; the other is flat in normal traffic and so invisible). The model's value is in the residual, not in being tuned toward known attacks.

Verdict: keep, deploy behind the rules, do not tune further.

Option 3 — A supervised classifier (random forest or gradient boosting) MEDIUM

Train on both normal and attack rows. Labels already exist in the dataset.

This would learn the unmatched-route feature in seconds and would post a very high number on our seven attacks. The problem is what that number would mean. It would be a classifier with a few hundred examples of each of seven attacks, and it would learn *those seven*. A new attack, or a small change to an old one, is outside its experience. The dataset design document rejected this on purpose: "an anomaly model fitted on attacks is a classifier with two examples of each attack, which is not what this is."

It also does nothing that the rules do not already do, at more cost and less explainability. About two days to build (a fork of `train.py`), but its success would be an illusion unless attack variants are held out — and we do not have variants.

Verdict: only if the project needs a headline number, and then say clearly what it measures.

Option 4 — Distance-based anomaly models (k-nearest-neighbour, LOF, autoencoder) MEDIUM

A different family of anomaly model that can, in principle, react to a feature that is flat in normal traffic.

These score a point by how far it sits from the training points, so a value of 0.54 in a column where training is all zero *does* register. In practice it is fiddly: the usual scaling step divides by that column's spread, which is zero; the score becomes dominated by whichever columns happen to be large; and results are harder to explain than a tree or a rule. Roughly a week to do properly, with an uncertain result — and the best case is matching what a one-line rule already does at zero false positives.

Verdict: not worth it while Option 1 exists.

Option 5 — A per-address suspicion score that builds up and decays MEDIUM

The general version of rules B and C. Each minute adds a little suspicion from any signal; suspicion fades with time; alert when it crosses a level.

This is the standard answer to "low and slow" attacks of any kind: a per-minute signal that is individually too weak to act on becomes decisive when it persists. Rules B and C are two hand-written instances of it. The general form catches slow variants nobody has written a rule for, at the cost of tuning a decay rate and keeping per-address state in Redis with expiry. Overlaps partly with what the control plane's campaign memory already does at a coarser level, so it should be designed with that in mind. Three to four days after Option 1 is in.

Verdict: good second step, once the rules are live and measured.

Option 6 — Sequence models over request streams (HMM, LSTM)**HIGH**

Would model the *shape* of an attack over time rather than one-minute snapshots. Needs far more data than we have, weeks of work, and produces decisions that cannot be explained line by line — which the design document requires for anything that blocks a person. Nothing in tonight's results suggests the problem needs this.

Verdict: no.

Option 7 — Ask a language model to classify traffic**REJECTED**

Already ruled out by the architecture, for a reason that has not changed: request paths and user-agents are written by the attacker, so anything that reads them into a language model can be told "this traffic is benign". The model stays outside the trust boundary and writes reports; it does not make decisions.

Verdict: no, and the reason is structural.

Option 8 — Loosen the false-positive budget or move the threshold**NO EFFECT**

Would not help. The scanning attacks score 0% at every threshold, because the model never learned the feature that separates them. No threshold fixes a feature the model is not using.

4. Side by side

Option	Complexity	Missed minutes caught	Normal minutes flagged	Explainable?	Holds on new attacks?	Verdict
1. Behavioural rules	LOW	65 / 76	0	Yes, line by line	Likely — each encodes what the attack must do	Do first
2. Keep Isolation Forest	LOW	10 / 76	3 (0.12%)	Partly (a score)	Yes, that is its purpose	Keep as layer 2
1 + 2 together	LOW	75 / 76	3 (0.12%)	Mostly	Likely	Recommended
3. Supervised classifier	MEDIUM	High on these 7 attacks	Low	Partly	No — learns these seven	Only for a headline
4. Distance models	MEDIUM	Unknown	Unknown	Poorly	Unknown	Not worth it
5. Suspicion accumulator	MEDIUM	≥ rules B + C	Tunable	Yes	Yes, by design	Step two
6. Sequence models	HIGH	Unknown	Unknown	No	Unknown	No
7. Language model	—	—	—	No	—	Rejected
8. Retune threshold	—	10 / 76	—	—	—	No effect

5. Recommended plan

- **Day 1 — scanner rules (A and B).** Extend `enumeration_path_traversal.go` : compute the unmatched-route share per address per minute, keep a five-minute rolling flag per address, emit evidence above the thresholds. Thresholds in config. Unit tests including the dead-link user, who must never trip it.
- **Day 2 — slow-login rule (C).** Extend `brute_force.go` with a per-address count of consecutive all-failed minutes, reset on any success. Threshold three, in config. A test built from the forgetful-user pattern (two failed minutes then success) must pass clean.
- **Day 3 — verify and write up.** The offline simulation is already done on v4 (this document). Run one ten-minute live collection round with the new detectors on and confirm the Go implementation flags the same minutes. Deploy the baseline v3 model behind the rules. Update `anomaly-model-results.md` with tonight's negative results and the rule numbers, so the next person does not repeat the feature-engineering detour.
- **Later, if slow variants appear that the rules miss:** Option 5.

6. Honest limits

- **The data is thin where it matters.** 76 missed minutes, 64 from one attack. The rules look perfect partly because there is not much to be imperfect on. The live confirmation round in the plan is there for this reason, and a second round with slightly varied attack scripts would be worth more than any further modelling.

- **The rules were written after looking at the attacks.** That is the same overfitting risk a classifier has. The defence is that each rule encodes something the attack has to do, not a detail of our scripts — but that is an argument, not a measurement, until a new attack variant is tried.
- **The 76 minutes were found with tonight's new metadata columns** (`detector_fired`, `gateway_blocked`). Both are needed: on the flood attack, 260 of 264 minutes were *blocked*, not missed, and the first column alone reports those as misses.
- **One scanner minute stays uncaught** by everything: three requests, all to real pages, no history yet. Two ways to catch it are lengthening rule B's window and Option 5; both trade a little delay for it.

7. Where the numbers come from

Dataset `datasets/v4` (spec v3, 14 features, 16,438 rows, 23 collection runs of 79 sessions each). Models in `models/v3-iforest`, `models/v3-iforest-regularity`, `models/v4-iforest`, each with an `evaluation.json` produced by `iasg.ml.evaluate`. "Missed minutes" are test-split rows with `label=1`, `detector_fired=0`, `gateway_blocked=0` and enough requests to be scored. Rule simulations join `features.csv`, `metadata.csv` and `quality.csv` on `row_id` and walk each client's minutes in order for the rolling and streak rules. The 0.0% figure for normal users is over all 121,480 benign requests in the raw captures; the per-minute benign counts (13,572) are scored benign rows across all splits.